Networks & Distributed Programming

The Transport Layer: TCP

Lecture 16



UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II



DIETI

UNINA

Transport Layer
TCP

TCP builds on the **reliable data transfer** mechanisms we have already studied: checksums, sequence numbers, acknowledgments, timers, and sliding windows.

- Like Go-Back-N, TCP uses cumulative acknowledgments.
- Like Selective Repeat, it can buffer out-of-order data.
- The key difference is that TCP numbers bytes, not packets.

Transport Layer
TCP

- TCP with respect to UDP is:
  - **Connection-oriented**: there is a “handshake” phase before the transmission that ensures the two processes (sender and receiver) to be “connected”.
  - **Reliable**: error detection, retransmissions, acknowledgments, timers, sequence numbers are implemented.
  - **Congestion and flow aware**: there is a regulation of the transmission rate depending on the receiver performance (flow) and the network performance (congestion).

- Connection orientation makes TCP full-duplex and point-to-point:
  - **Full-duplex**: if host A is connected with B, then B is connected with A.
  - **Point-to-point**: single sender and single receiver, i.e., the transfer of data from one sender to many receivers is not possible.
    - UDP allows **multicasting**

- TCP is also oriented on sending/receiving data streams:
  - Multiple TCP segments may be part of a bigger data stream (ordered sequence of data).
  - Single UDP datagrams are considered mostly decoupled.

Transport Layer
TCP

A logical connection

The two transport modules behave as if a reliable channel joined them directly.

Underneath, the segments cross a network that only routes.

- It is not a circuit: no capacity is reserved, and nothing is set aside for the connection inside the network.
- TCP connection state — buffers, variables, and timers — is maintained at the end hosts, not by IP routers.
- Routers forward IP datagrams based on IP headers, without needing to interpret TCP connection state
- Which is why reliability is built here. The network layer is busy deciding where a packet goes, not whether it arrived.

TCP: RFC 793 (1981), superseded by RFC 9293 (2022)

Transport Layer
TCP Buffers

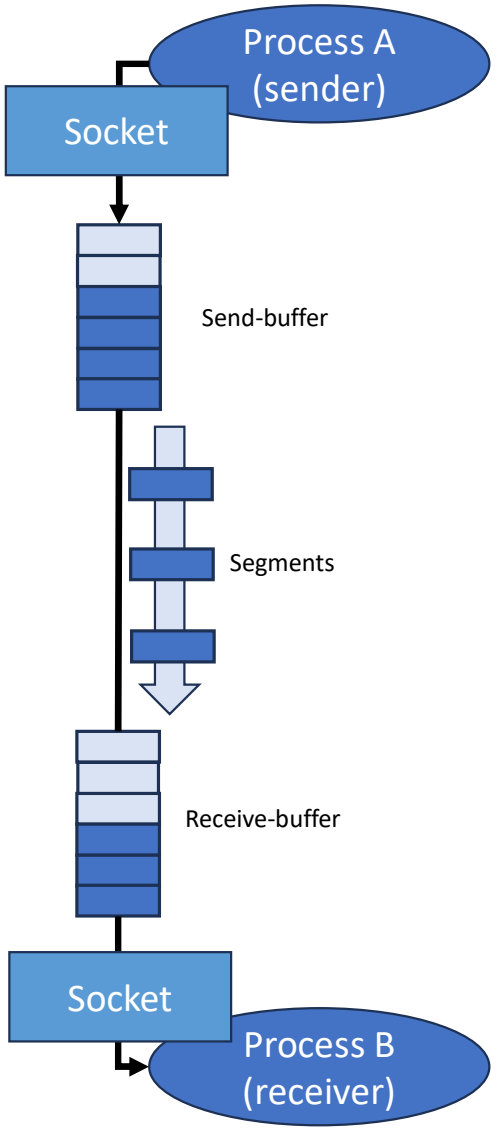
In TCP connection sending and receiving operations strongly **rely on buffers** to be performed.

- write does not transmit: it copies bytes into the send buffer, and TCP sends them from there at its own convenience.
- read does not receive: it takes what is already sitting in the receive buffer, put there by earlier segments.

Buffers allow us to **partially decouple** transmission times from:

- **Application delays**.
- **OS delays** in multiplexing demultiplexing packets.
- **Network delays** (oscillations of network performance).

There is also the limit of the transmission rate, so long messages could **be broken into smaller segments** that may be collected in buffers before being disassembled (sender-side) or reassembled (sender-side).



Transport Layer

TCP: Maximum Segment Size

- The amount of **data inside a segment** is limited by the **Maximum Segment Size** (MSS). E.g., in IPv4 we may have:

$$MSS + 40 = MTU$$

- Where:
  - The **Maximum Transmission Unit** (MTU) is the maximum length of a frame acceptable by the link-layer (e.g., in Ethernet it is 1500 bytes).
  - The **combined header size** of TCP and IP headers (typically 20+20 bytes).

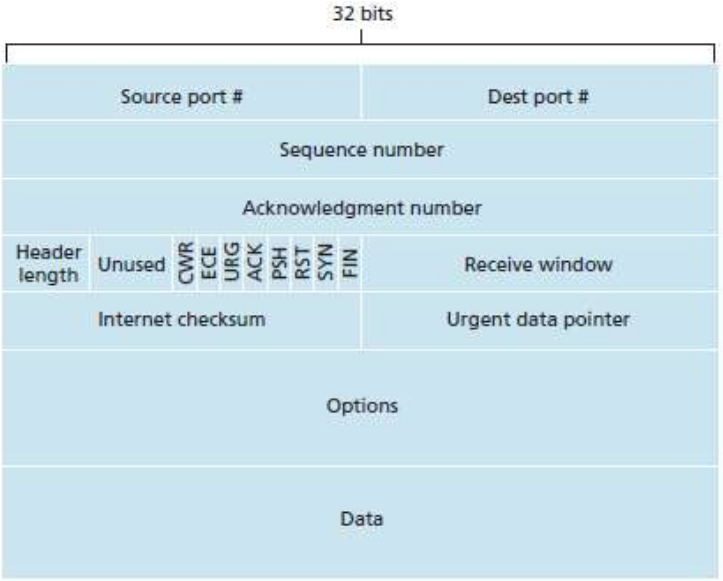
- **On sending**: the segments are created from the application data and passed down to the network layer, where they are separately encapsulated within network-layer IP datagrams to be sent.
- **On receiving**: the data is placed into the receive-buffer, the application grabs data from the buffer when ready.

E.g., three-megabyte image is not a message: it is two thousand segments, and the application never learns where the cuts fell.

Transport Layer

TCP Segment

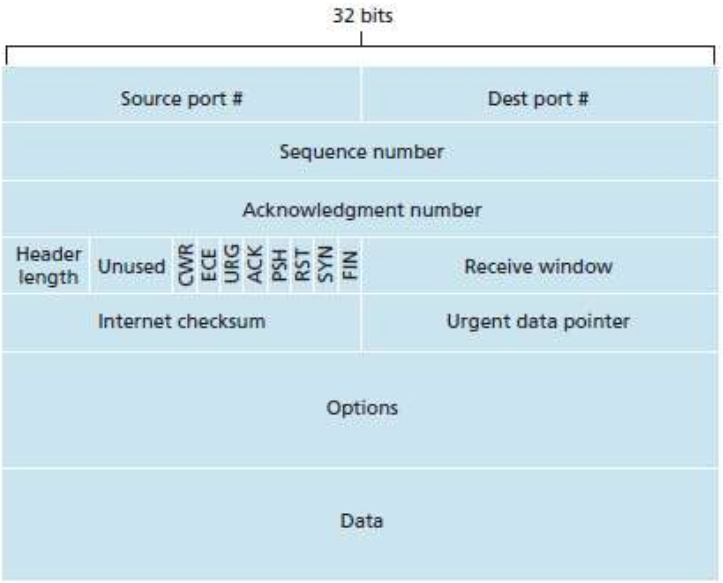
- Two port numbers, as in UDP: the segment is still demultiplexed onto a socket.
- Two 32-bit fields, sequence and acknowledgment number, which carry the reliable transfer.
- A 16-bit receive window: how many bytes the other side will accept.
- Header length in 32-bit words, because the options make the header variable.
- One checksum, computed as it was for UDP.



Transport Layer

TCP Segment

- The **header** contains several fields:
  - **Sequence number** (32-bit) for reliable data transfer.
  - **Acknowledgment number** (32-bit) for reliable data transfer.
  - **Receive window** (16-bit) used to indicate the number of bytes that a receiver is willing to accept (for flow control).
  - **Header length** (4-bit) specifies the number of 32-bit words contained into the header. Note that **TCP header is variable due to the options** field.
  - **Options field** (K-bit) used for optional processes such as negotiating the MSS, time-stamping (RTT), etc.
  - **Flags** (6/12-bit) including:
    - ACK bit indicates that **this segment is an ACK** packet (so the acknowledgment number field is in use).
    - RST, SYN, and FIN bits used for **connection setup and teardown**.
    - CWR and ECE bits used in explicit congestion notification (optional).
    - PSH bit tells the receiver to pass the data to the application immediately. (RARE)
    - URG bit indicates that the segment contains “urgent” data. (RARE)
  - **Checksum** (16-bit): for integrity check.
  - **Urgent data pointer field** (16-bit) indicates the location of the last byte of the urgent part of the data. (RARE)

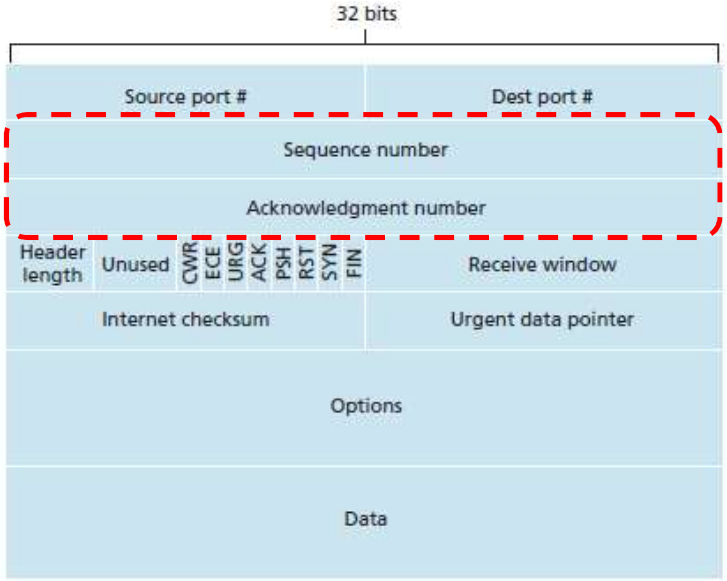


Transport Layer

TCP Sequence and Acknowledgment Numbers

- Sequence numbers are used not only for reliable data transfer but **also to manage segmentation**.
  - If a large data stream is transmitted, we need **to break it into smaller "pieces"** depending on the MSS.
  - Each "piece" of the stream is put into a TCP segment.

- Sequence numbers and acknowledge numbers are **strictly related**.
  - The **sequence number** of a segment is **the position that the first byte of the segment's data has into the data stream**.
  - The **acknowledgment number** is the **next piece of the data stream** we are expecting.

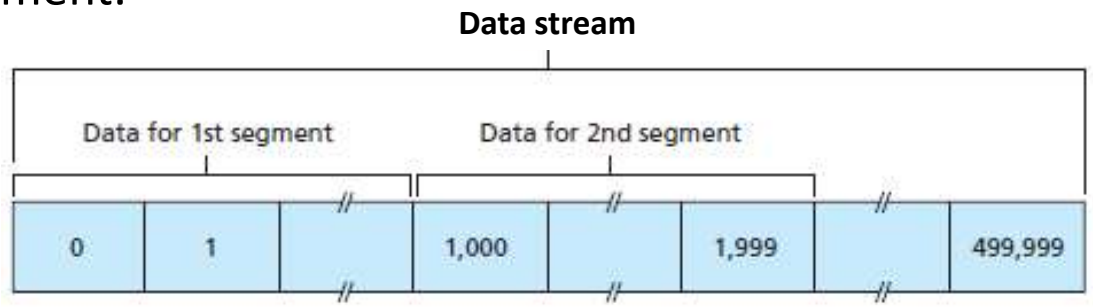


Note: the numbering survives a different choice of where to cut the stream.

Transport Layer

TCP Sequence and Acknowledgment Numbers

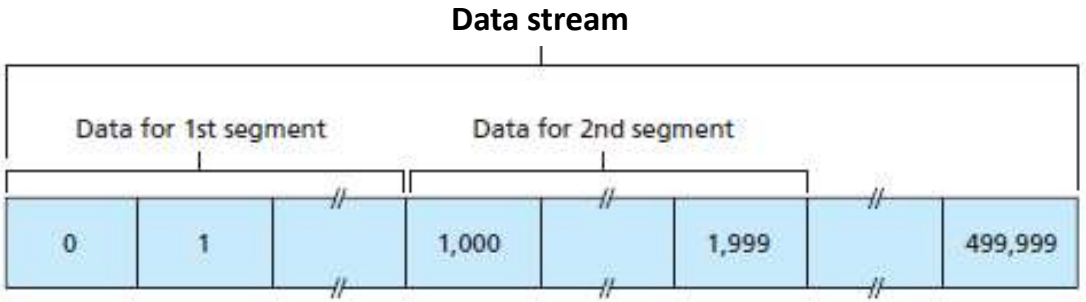
- Notice that TCP is a general-purpose transport protocol, it has no information about the data to be transmitted. From TCP's viewpoint data are **just an ordered stream of bytes**.
- The **sequence number** for a segment is then the byte-stream **number of the first byte of the data** in the segment.



- Example: assume a **data stream of 500,000 bytes** (~500KB), and a **MSS of 1,000 bytes**.
- The TCP constructs **500 segments** where the first segment has sequence num. 0, the second segment has sequence num. 1000, the third segment has sequence num. 2000, and so on.

Transport Layer

TCP Sequence and Acknowledgment Numbers



- Let's assume this data stream to be **passed from host A to host B**.
- For simplicity, let's **neglect previous interaction** (data exchange from connection establishment), so **we are starting from sequence number 0**:
  - The receiver will get a first segment of 1,000 bytes, i.e., from **sequence number 0 to sequence number 999**
  - The receiver will then acknowledge the transmission by setting an **acknowledgment number of 1,000** (next expected byte in the stream).
  - The receiver will get the next segment of 1000 bytes, i.e., from **sequence number 1,000 to sequence number 1,999** and so on…

Note: The acknowledgment number is the next expected byte in the stream, and therefore confirms the receipt of all preceding bytes.

Transport Layer

TCP Cumulative acknowledgement

One number for everything received

A receives bytes 0–535 from B, and then a segment with bytes 900–1000. Bytes 536–899 never arrived. A acknowledges 536.

- An ACK confirms that all bytes below this number have been received in order, but does not acknowledge bytes beyond the gap.
- A single cumulative ACK can acknowledge multiple segments. If an ACK is lost, a later cumulative ACK can still confirm the previously received data.
- The cumulative ACK alone cannot report out-of-order bytes. The SACK option allows TCP to report which additional blocks have been received.
- A missing segment prevents the cumulative ACK from advancing until the gap is filled.

Transport Layer

TCP: Out-of-order segments: keep or discard?

Bytes 900–1000 have arrived, but bytes 536–899 are missing. Should the receiver keep the out-of-order bytes?

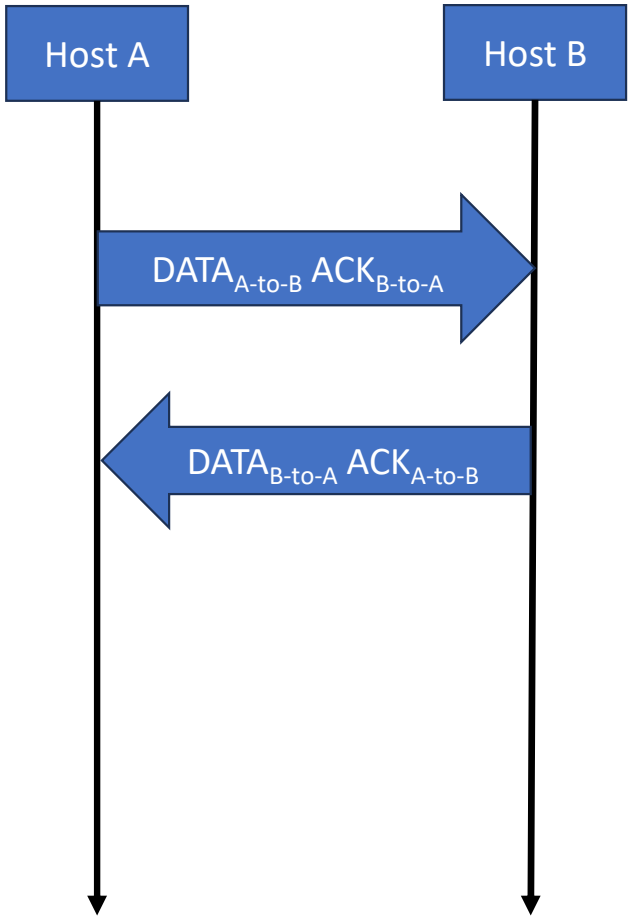
- TCP recommends buffering out-of-order data, but does not strictly require it.
- Discard: simplifies receiver processing, but may require retransmitting data that already arrived.
- Keep: stores out-of-order bytes until the missing data arrives.
- SACK: allows the receiver to report which additional bytes have arrived, helping the sender retransmit only missing data (Selective Acknowledgment).

Transport Layer

TCP Sequence and Acknowledgment Numbers

- What happens if the **receiver is sending data in return**?
  - Remind: TCP communication is full-duplex, if 2 hosts, A and B, communicate there are also 2 flows of data to be considered for reliable data transfer: **A-to-B** and **B-to-A**.

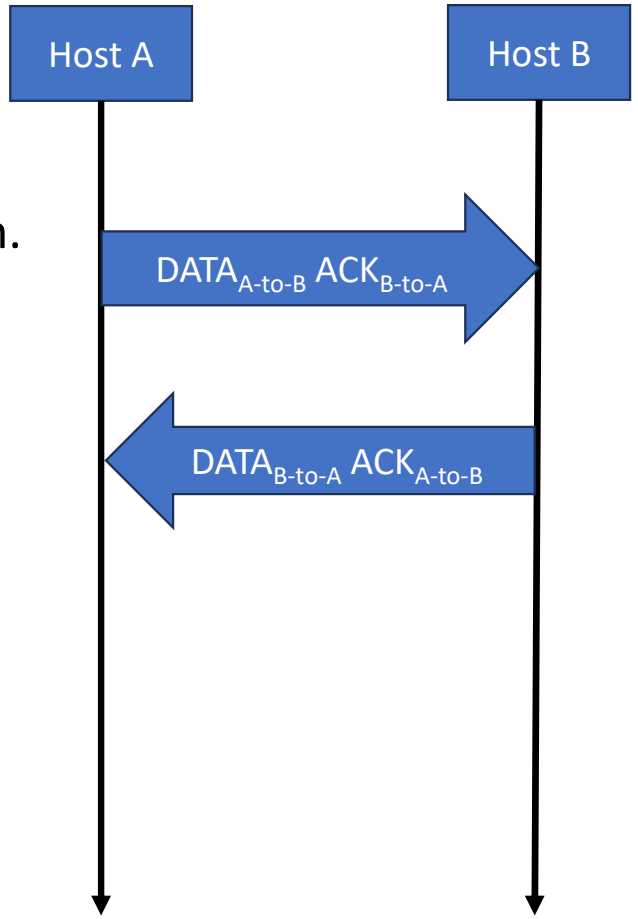
- In this case we can **use sequence numbers and acknowledgment numbers at the same time**:
  - Segments from A have sequence numbers related to A-to-B data stream and acknowledge numbers related to the B-to-A data stream.
  - Segments from B have sequence numbers related to B-to-A data stream, and acknowledge numbers related to the A-to-B data stream.



Transport Layer

TCP Sequence and Acknowledgment Numbers

- **One connection, two byte streams:** A to B and B to A, numbered independently of each other.
- A segment leaving A carries a sequence number for the A-to-B stream and an acknowledgment number for the B-to-A stream.
- The two fields in one header therefore speak about two different streams.
- An acknowledgement travelling inside a segment that also carries data is said to be piggybacked: it costs no extra packet.

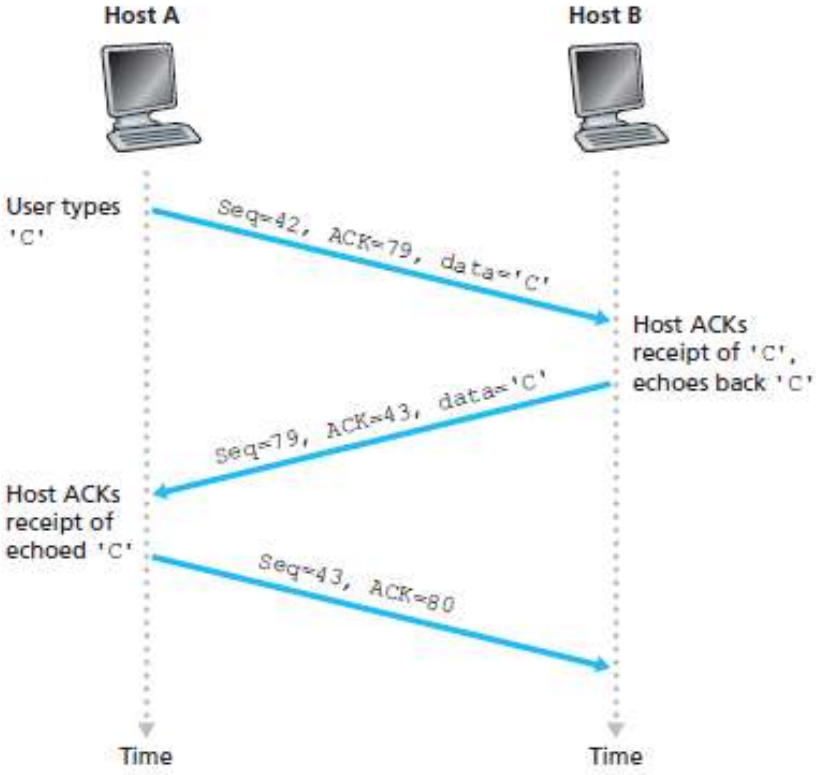


Transport Layer

TCP Sequence and Acknowledgment Numbers

- This is a simplified example of two hosts sending data each other.
  - We are considering 2 messages of **just 1 byte** (1 char) form A to B and vice versa.
  - Specifically, host A transmits a ‘c’ **that is echoed back** by B.

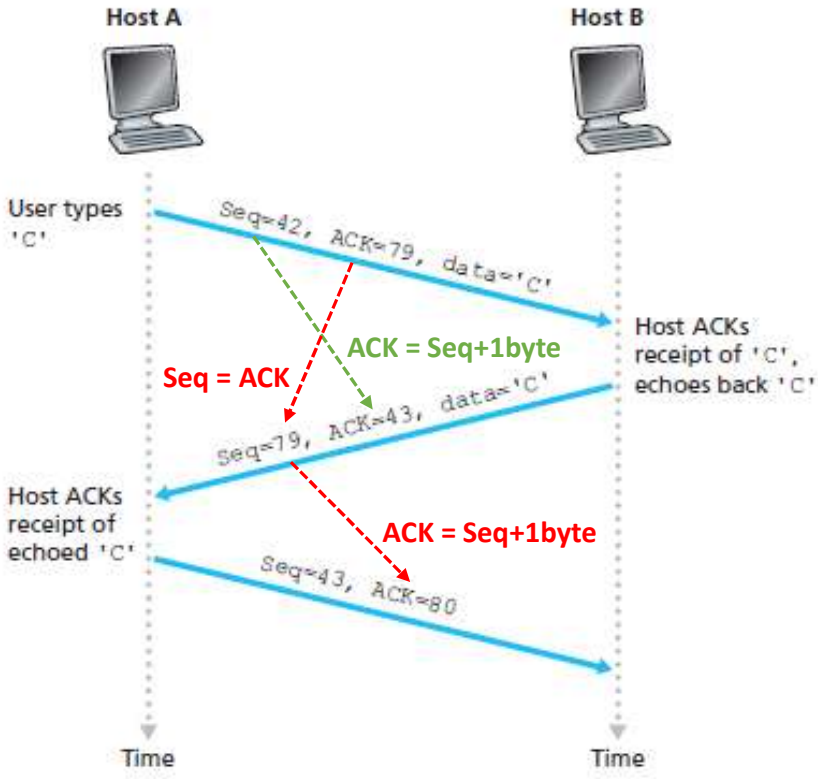
- Here that segments provide **ACK and DATA at the same time** (so the ACK flag is 1).



Transport Layer

TCP Sequence and Acknowledgment Numbers

1. From A to B:
   - The ack number as the seq number of the next expected packet in the B-to-A flow (byte 79).
   - The **seq number** as the position of **the single byte** we are transmitting (byte 42).
2. From B to A:
   - The ack number as the seq number of the next expected packet in the A-to-B flow (i.e., seq + 1 byte of the ‘c’ char).
   - The **seq number** as the position of **the byte we are transmitting** (byte 79).
3. From A to B:
   - Pure **ACK segment** is sent, having seq number 43 (as expected by B) and ack number 80 (next expected).



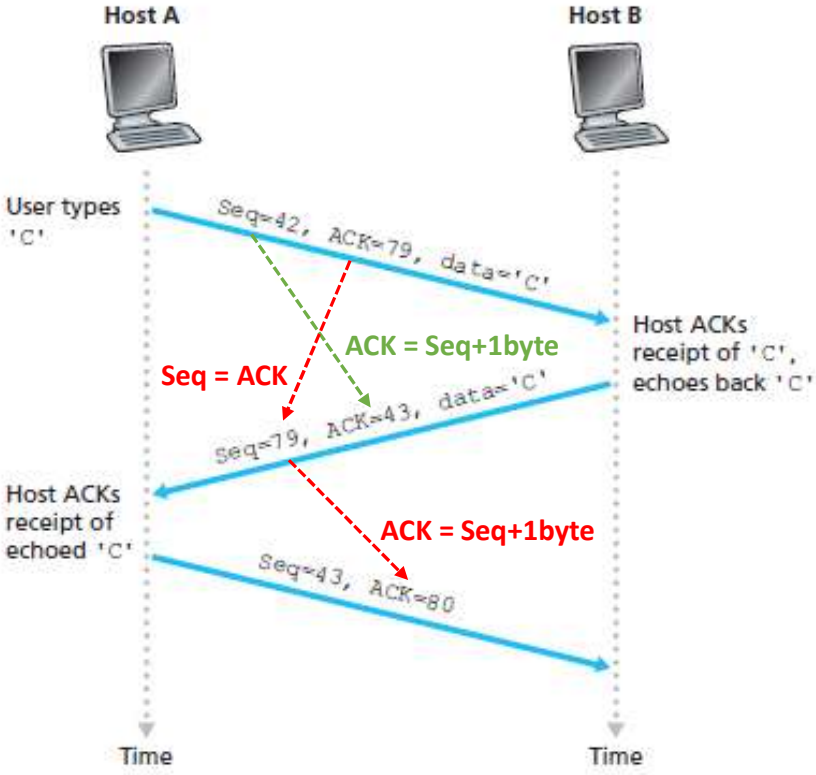
A-to-B info are in green.
B-to-A info are in red.

Transport Layer

TCP Sequence and Acknowledgment Numbers

The third segment carries nothing. What is seq 43 doing there?

- A Pure ACK carries no application data (DATA = 0), but still includes a sequence number.
- Since no data is transmitted, A's sequence number does not advance. It remains 43.
- A Pure ACK does not require acknowledgment and does not start a retransmission timer.
- If the ACK is lost, it is not retransmitted automatically. A later ACK can carry the same or updated acknowledgment number.



A-to-B info are in green.
B-to-A info are in red.

Transport Layer

TCP Sequence and Acknowledgment Numbers

Instructions: Assume that the TCP connection has already been established, all segments arrive correctly and in order, and no additional data is exchanged unless explicitly stated.

Exercise 1

Host A and Host B have an active TCP connection. Host A sends the following segment to Host B:

A → B

SEQ = 150

ACK = 400

DATA = 20 bytes

Questions:

If Host B responds with a pure ACK segment (no data), what will its SEQ and ACK numbers be?

Transport Layer

TCP Sequence and Acknowledgment Numbers

Instructions: Assume that the TCP connection has already been established, all segments arrive correctly and in order, and no additional data is exchanged unless explicitly stated.

Exercise 1

Host A and Host B have an active TCP connection. Host A sends the following segment to Host B:

A → B

SEQ = 150

ACK = 400

DATA = 20 bytes

Questions:

If Host B responds with a pure ACK segment (no data), what will its SEQ and ACK numbers be?

SEQ = 400, ACK = 170, DATA = 0

Transport Layer

TCP Sequence and Acknowledgment Numbers

Exercise 2

Host A sends a TCP segment containing 50 bytes to Host B:

A → B

SEQ = 1000

ACK = 3000

DATA = 50 bytes

Host B successfully receives the segment and responds by sending 30 bytes of data together with an acknowledgment.

Host A then receives Host B's segment and replies with a pure ACK.

Questions:

What are the SEQ and ACK numbers of the segment sent by Host B?

What are the SEQ and ACK numbers of the final segment sent by Host A?

Transport Layer

TCP Sequence and Acknowledgment Numbers

Exercise 2

Host A sends a TCP segment containing 50 bytes to Host B:

A → B, SEQ = 1000, ACK = 3000, DATA = 50 bytes

Host B successfully receives the segment and responds by sending 30 bytes of data together with an acknowledgment.

Host A then receives Host B's segment and replies with a pure ACK.

Questions:

What are the SEQ and ACK numbers of the segment sent by Host B? SEQ = 3000, ACK = 1050, DATA = 30

What are the SEQ and ACK numbers of the final segment sent by Host A? SEQ = 1050, ACK = 3030, DATA = 0

Transport Layer

TCP Sequence and Acknowledgment Numbers

Exercise 3

Host A and Host B have an established TCP connection. The next sequence number available to Host A is 500, while the next sequence number available to Host B is 2000.

The following communication takes place:

1. Host A sends 100 bytes to Host B.
2. Host A sends another 200 bytes without waiting for an ACK.
3. Host B receives both segments and sends a single cumulative ACK.

Complete the table:

Segment	SEQ	ACK	DATA
1. A → B	500	2000	100 bytes
2. A → B	?	?	200 bytes
3. B → A	?	?	0 bytes

Transport Layer

TCP Sequence and Acknowledgment Numbers

Exercise 3

Host A and Host B have an established TCP connection. The next sequence number available to Host A is 500, while the next sequence number available to Host B is 2000.

The following communication takes place:

1. Host A sends 100 bytes to Host B.
2. Host A sends another 200 bytes without waiting for an ACK.
3. Host B receives both segments and sends a single cumulative ACK.

Complete the table:

Segment	SEQ	ACK	DATA
1. A → B	500	2000	100 bytes
2. A → B	600	2000	200 bytes
3. B → A	2000	800	0 bytes

TCP uses independent sequence numbers for each direction. Host A sends 300 bytes (500–799), so Host B replies with a cumulative ACK = 800. Host A's ACK remains 2000 because it has not received any new data from Host B.

Transport Layer

Where the numbering starts

- In every example so far the first byte was numbered 0. In a real connection it never is.
- Each side picks its initial sequence number at random, and tells the other one while the connection is being opened.
- The reason is the one we met with sequence number reuse: a segment from an earlier connection can still be alive in the network.
- Same two hosts, same two ports, numbering from zero every time — and an old copy would be accepted as current data.
- A random start does not make that impossible. It makes it improbable, which is as far as the protocol can go.

Transport Layer

The sender in three events

Event	What the sender does
Data arrives from the application	Build a segment with sequence number `NextSeqNum`, start the timer if it is not already running, pass it to IP, and advance `NextSeqNum` by the number of bytes sent.
The timer expires	Retransmit the unacknowledged segment with the smallest sequence number, and start the timer again.
An acknowledgement y arrives	If $y >$ `SendBase`, move `SendBase` to y, and restart the timer if anything is still unacknowledged.

Two variables hold the whole sender: SendBase, the oldest byte not yet acknowledged, and NextSeqNum, the next one to go out.

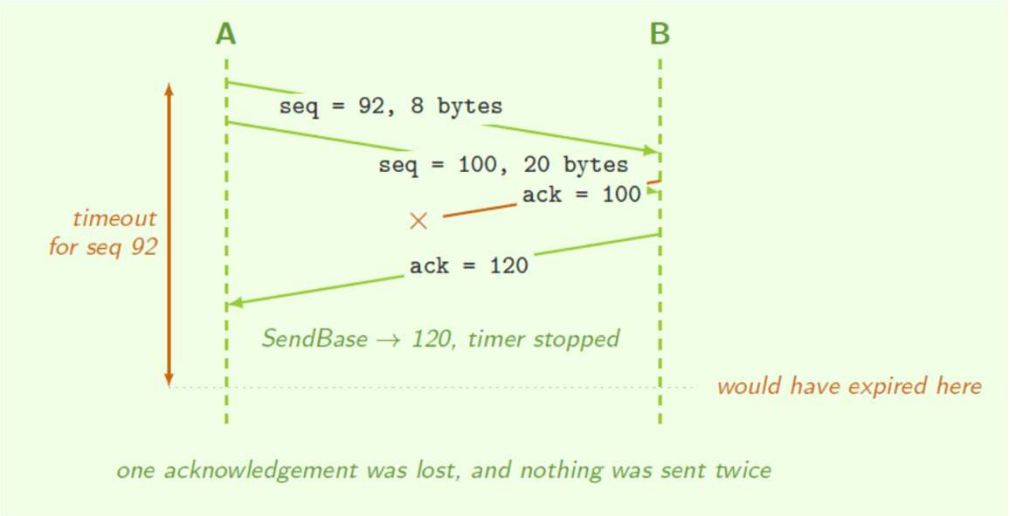
Transport Layer

One timer, not one per segment

- A timer for every outstanding segment is the natural design, and it is what selective repeat asks for.
- In the basic TCP model, a single retransmission timer tracks the oldest unacknowledged segment.
- When it fires, only that segment goes out again, however many others are in flight and unanswered.
- When the retransmission timer expires, TCP retransmits the oldest unacknowledged segment. Further retransmissions may be needed if other segments were also lost.
- Using a single retransmission timer reduces the overhead of managing multiple timers, especially when many connections are active.

Transport Layer

One timer, not one per segment



The first acknowledgement was lost, and nothing was retransmitted: ack 120 says that everything below 120 arrived, which covers both segments at once. SendBase jumps straight to 120 and the timer is stopped.

Transport Layer

Doubling the timeout

- When the timer expires, TCP retransmits and sets the next timeout to twice the interval that has just run out.
- Three quarters of a second becomes 1.5, then 3, then 6: the waits grow exponentially for as long as nothing comes back.
- The estimate is not used while this is going on. It comes back as soon as an acknowledgement arrives and the timer restarts normally.
- A timeout usually means congestion, and retransmitting harder into a congested network makes it worse.
- So the sender backs off. It is a crude form of congestion control, and the same idea returns when stations collide on a shared link.

Transport Layer
Retransmission Timeout

- **TCP reliable data transfer mechanism makes extensive use of timeouts**.

- The default approach in pipelining-based TCP transmission is to **retransmit segments only if the associated ACKs are not received before the timeout**.

- Therefore, **timeout estimation is critical** as it strongly affects the performance of the transmission.
  - Too long timeout causes communication to be slowed.
  - Too short timeout causes the packets to overlap (and unnecessary packets to be retransmitted, producing useless traffic on the network).

Transport Layer

Retransmission Timeout

- Too long: every loss is paid with a long idle wait, and the transfer stops for as long as the timer runs.
- Too short: segments that are merely slow get declared lost, and the network carries copies nobody needed.
- We have already seen the extreme: with a timeout shorter than the round trip, everything is retransmitted, losing nothing.
- So the timeout has to be larger than the RTT — and the RTT is not a constant, and nobody tells the sender what it is.

Transport Layer

Retransmission Timeout

- **SampleRTT**
  - The time between passing a segment to IP and receiving an acknowledgement that covers it. One measurement, for one segment: $t_{ack} - t_{seq}$
  - TCP measures one segment at a time, not all of them: roughly one new sample per round trip.
  - The samples move, and by a lot — router queues fill and drain, and the host at the other end has a load of its own.
  - So a single sample is a poor estimate of the round trip, and a very poor basis for a deadline.
  - Segments that have been retransmitted are never sampled at all.

Transport Layer

RTT Estimation

- To suitably set timeouts **we need a round-trip time (RTT) estimation**.
  - The timeout, i.e., the time to wait for a segment's ACK, should be **larger than RTT** otherwise unnecessary retransmissions would be sent.

- TCP estimates this value by **sampling the RTT of successfully acknowledged segments** that have not been retransmitted (one-shot).
- Since the time of a sampled RTT (SampleRTT) **may fluctuate** (due to traffic congestion, load of the receiver, etc.) the RTT estimation (EstimatedRTT) is given by the **exponential weighted moving average (EWMA)**:

$$\text{EstimatedRTT} = (1-\alpha) \cdot \text{EstimatedRTT} + \alpha \cdot \text{SampleRTT}$$

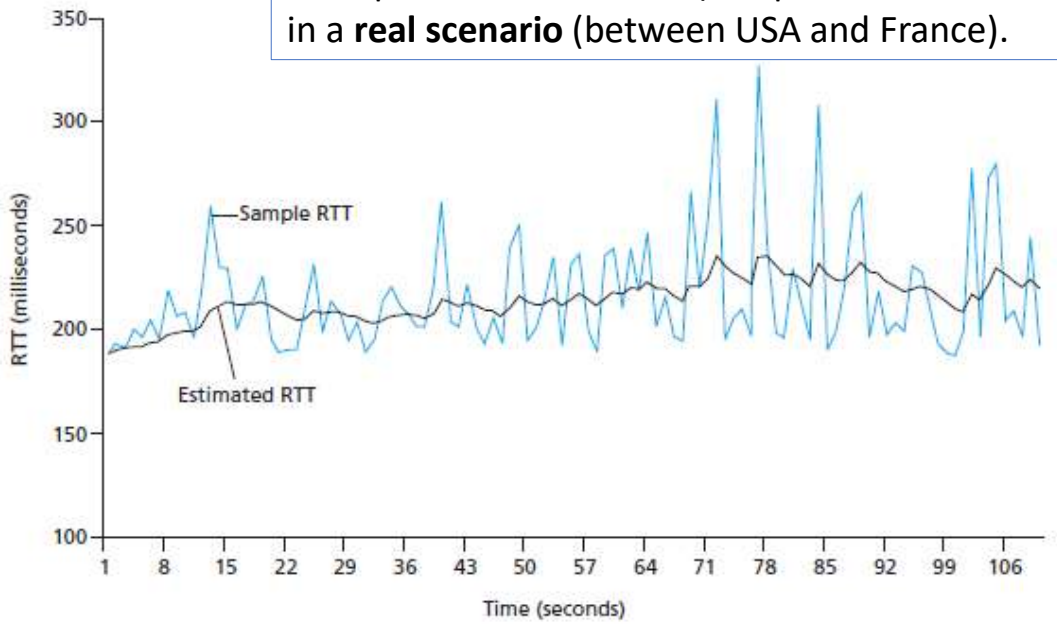
Where α is a parameter which is usually *0.125* (i.e., *1/8*).

Transport Layer
RTT Estimation

- We can also consider the variability of the RRT in our timeout calculation.

- The **variability of the RTT** (DevRTT) can be given by the difference between the sample (SampleRTT) and the estimation (EstimatedRTT):

Example of RRT behavior (sampled and estimated) in a **real scenario** (between USA and France).



$$DevRTT = (1-\beta) \cdot DevRTT + \beta \cdot | SampleRTT - EstimatedRTT |$$

Where β is a parameter which is usually *0.25* (i.e., *1/4*).

Transport Layer
Retransmission Timeout

- Having these estimations, **it is possible to define a retransmission timeout** for TCP that is not too much lower or higher than the estimated RTT:

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \cdot \text{DevRTT}$$

- Here, the deviation of the RTT is used to set a **reasonable and adaptive margin** from the estimated RTT.
  - It increases when RTT oscillates, so the timeout window is larger **when we are uncertain** about our current RTT.

- By default, the **initial value** of the RTT (at t=0) is 1 second.

Transport Layer

TCP Acknowledgments: Delayed ACK and Duplicate ACK

What arrives	What the receiver does
In-order segment, everything below it already acknowledged	Wait up to 500 ms for the next one; if it does not come, acknowledge.
In-order segment, with one earlier segment still waiting to be acknowledged	Acknowledge both at once, with a single number.
Out-of-order segment, numbered higher than expected: a gap	Acknowledge immediately, repeating the number at the bottom of the gap.
A segment that fills the gap	Acknowledge immediately, up to the end of what is now contiguous.

- **Delayed acknowledgement:**
  - Wait briefly for a second segment (1). If it arrives, acknowledge both (2); otherwise, acknowledge the first.
- **Duplicate ACK:**
  - Immediately repeat the ACK for the first missing byte (3). When the gap is filled, advance the cumulative ACK to the next missing byte (4)

Transport Layer

TCP Acknowledgments: Duplicate ACK

- A duplicate acknowledgement repeats a number the sender already had: taken alone, it carries no new information.
- But it was sent for a reason. Something arrived at the receiver, and it was not what the gap needed.
- So each duplicate is evidence that segments after the missing one are getting through: the path works, one segment does not.
- A sender that waits for its timer here is waiting for a deadline whose answer it already has.
- TCP has no negative acknowledgement. A repeated number is the closest thing to one, and it costs no new packet type.

Transport Layer
Fast Retransmit

- Timeout-triggered retransmission is quite effective, but the **timeout period can be relatively long**.

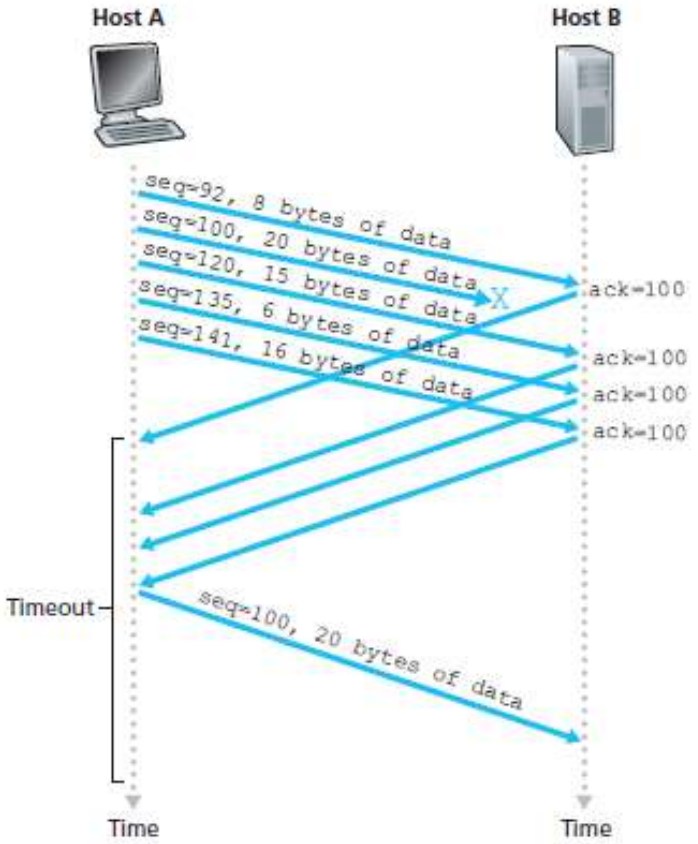
- **Fast retransmit** is an approach in which the receiver signals the sender that one packet might be lost by sending **duplicate ACKs**.
  - When a TCP receiver receives a segment with a **sequence number that is larger than the expected one** it means that a segment has been reasonably missed.
  - The receiver **resends the old ACK** (duplicate ACK) having as ACK number the one of the expected segment.
  - If the **sender receives N duplicates** (typically 3 duplicates), it assumes that the previous segment has been lost so it is (fast) retransmitted **before the deadline**.

Transport Layer

Fast Retransmit

- In this example we have a lost segment with sequence number 100.
  - Every time host B receives an out-of-order segment it **sends back a duplicated ACK** (ack=100).
  - When host A **receives 3 duplicates**, it assumes segment 100 to be lost and **resends it** (fast retransmit).
  - When host B finally receives the lost segment**, it will send the ACK of the next expected segment** depending on its own policy (go-back-N or selective repeat).

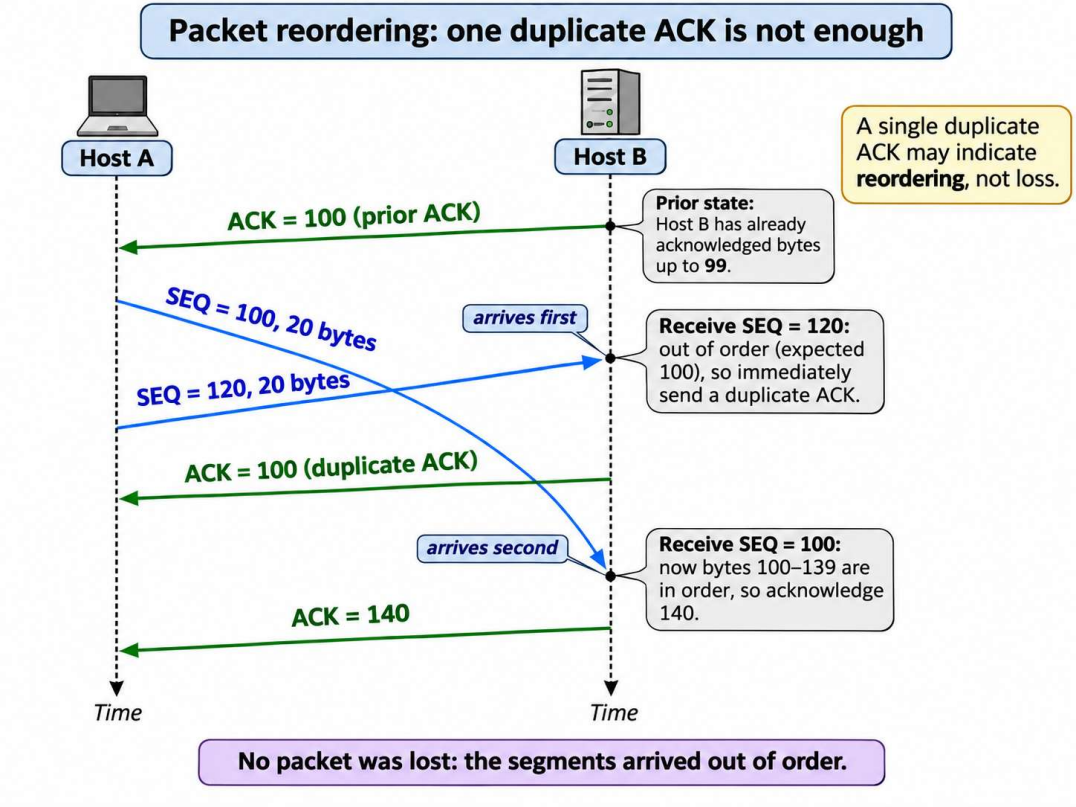
- But why don't we retransmit segments directly when the first duplicated ACK is received?
  - Because **we may receive wrong ACKs for different reasons** (not only if segments are lost).
  - For example, if **2 segments are received in the wrong order** (swapped during transmission), the **receiver will send 1 duplicate ACK** (for the first segment) before detecting the swap.



Transport Layer

Fast Retransmit

For example, if **2 segments are received in the wrong order** (swapped during transmission), the **receiver will send 1 duplicate ACK** (for the first segment) before detecting the swap.



Transport Layer

Fast Retransmit

… but we may have a similar case with 3 ack:

Host A sends four TCP segments, each containing 20 bytes:

Segment	SEQ	Arrival at B
1	100	4th (delayed)
2	120	1st
3	140	2nd
4	160	3rd

What happens?

Transport Layer

Fast Retransmit

… but we may have a similar case with 3 ack:

Host A sends four TCP segments, each containing 20 bytes:

Segment	SEQ	Arrival at B
1	100	4th (delayed)
2	120	1st
3	140	2nd
4	160	3rd

After receiving 3 Duplicate ACKs, Host A triggers Fast Retransmit of SEQ = 100.
However, the original segment was only delayed, not lost. When B receives it, it sends ACK = 180.

Three Duplicate ACKs may trigger an unnecessary (spurious) retransmission. TCP still delivers the data correctly, but additional bandwidth is wasted.

Transport Layer

Go-Back-N or selective repeat?

- Like Go-Back-N: cumulative acknowledgements, no individual answer for what arrives early, one timer and two variables.
- Unlike Go-Back-N: a timeout retransmits one segment, not everything outstanding — and not even that, if the next acknowledgement covers it.
- Like selective repeat: the receiver buffers what arrives early instead of throwing it away.
- So TCP is neither: cumulative acknowledgements over a receiver that keeps things, which is the useful half of each.
- An option, selective acknowledgement, lets the receiver name the blocks it is holding — and moves TCP the rest of the way.

Transport Layer
Summary

- A TCP connection is state in **two end systems**: buffers, variables, numbers. The network in between knows nothing of it.
- **Sequence numbers** count bytes
- An **acknowledgement** names the next byte expected, and everything below it has arrived.
- **The sender keeps one timer**, on the oldest unacknowledged segment, its length is estimated from the traffic itself.
- **Duplicate acknowledgements** say that a segment is missing
- After N acknowledge of old segments there is a **fast retransmit**.

- Still open: how the two ends agree on their initial numbers, and what the receive window in the header is for.